

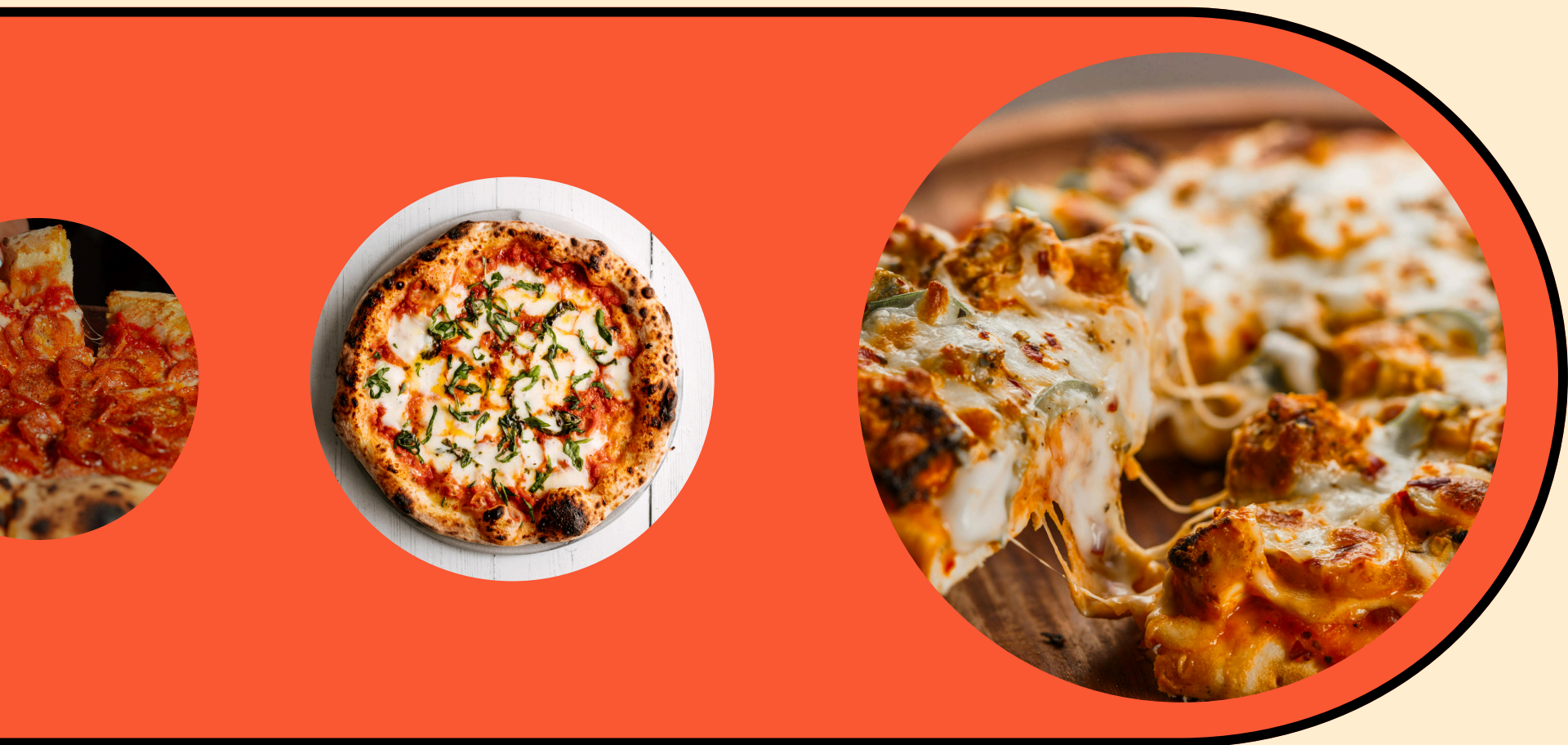
KEEP PIZZA



PIZZA SALES

DISCOUNT • DISCOUNT • DISCOUNT • DISCOUNT • DISCOUNT •
25%





MEET THE CEO

Hello, my name is Md.Shahil Khan I am an aspiring Data Analyst with knowledge of SQL, Python, Excel, and Power BI. I enjoy solving problems, analyzing data, and turning data into meaningful business insights. I am continuously improving my skills and looking forward to starting my career in data analytics.

I CREATED THE DATABASE TABLES MANUALLY BY DEFINING THE REQUIRED COLUMNS AND DATA TYPES. SINCE THE ORIGINAL DATASET WAS LARGE.

```
1 • ⊖ create table orders (  
2     order_id int not null,  
3     order_date date not null,  
4     order_time time not null  
5     );  
6  
7 • ALTER TABLE orders ADD PRIMARY KEY (order_id);  
8  
9 • ⊖ create table order_details (  
10    order_details_id int not null,  
11    order_id int not null,  
12    pizza_id text not null,  
13    quantity int not null,  
14    primary key(order_details_id)  
15    );
```

BASIC

PROBLEM AND QUERY

```
-- 1. Retrieve the total number of orders placed.  
select count(order_id) as total_orders from orders;
```

PROBLEM AND QUERY

```
4      -- 2. Calculate the total revenue generated from pizza sales.
5  ●    select
6  ⊖    round (
7        sum(order_details.quantity*pizzas.price),
8        2 )as total_revenue
9
10     from
11     order_details join
12     pizzas on
        order_details.pizza_id = pizzas.pizza_id;
```

PROBLEM AND QUERY

```
14      -- 3. Identify the highest-priced pizza.  
15  ●   SELECT pizza_id, pizza_type_id, price AS highest_priced_pizza  
16      FROM pizzas  
17      ORDER BY price DESC  
18      LIMIT 1;
```

```
90      -- 4. Identify the most common pizza size ordered.
91  ●    SELECT
92          p.size,
93          SUM(od.quantity) AS total_pizzas_sold
94  FROM pizzas p
95  JOIN order_details od
96      ON p.pizza_id = od.pizza_id
97  GROUP BY p.size
98  ORDER BY total_pizzas_sold DESC;
```

PROBLEM AND QUERY

```
30      -- 5. List the top 5 most ordered pizza types along with their quantities.
31  ●    SELECT
32          pizza_type_id, SUM(quantity) AS quantity
33  FROM
34      order_details
35      JOIN
36      pizzas ON pizzas.pizza_id = order_details.pizza_id
37  GROUP BY pizzas.pizza_type_id;
```

INTERMEDIATE

PROBLEM AND QUERY

```
-- 1. Join the necessary tables to find the total quantity of each pizza category ordered.  
SELECT  
    pt.category, SUM(od.quantity) AS total_quantity  
FROM  
    order_details AS od  
    JOIN  
    pizzas AS p ON p.pizza_id = od.pizza_id  
    JOIN  
    pizza_types AS pt ON p.pizza_type_id = pt.pizza_type_id  
GROUP BY pt.category;
```

PROBLEM AND QUERY

```
12      -- 2. Determine the distribution of orders by hour of the day.  
13  ●    SELECT  
14          HOUR(order_time) AS order_hour,  
15          COUNT(order_id) AS total_order  
16      FROM  
17          orders  
18      GROUP BY HOUR(order_time);
```

PROBLEM AND QUERY

```
20      -- 3. Join relevant tables to find the category-wise distribution of pizzas.
21 •    SELECT
22         category, COUNT(name)
23     FROM
24         pizza_types
25     GROUP BY category;
```

PROBLEM AND QUERY

```
27      -- 4. Group the orders by date and calculate the average number of pizzas ordered
28  ●    SELECT
29          round(AVG(order_by_date.quantity),0) as average_no_of_pizzas_opd
30  FROM
31  ⊖    (SELECT
32          orders.order_date, SUM(order_details.quantity) AS quantity
33      FROM
34          orders
35      JOIN order_details ON orders.order_id = order_details.order_id
36      GROUP BY orders.order_date) AS order_by_date;
```

PROBLEM AND QUERY

```
38      -- 5. Determine the top 3 most ordered pizza types based on revenue.
39  ●    SELECT
40          pizzas.pizza_type_id,
41  Ⓣ    ROUND(SUM(order_details.quantity * pizzas.price),
42          2) AS revenue
43  FROM
44      pizzas
45      JOIN
46      order_details ON pizzas.pizza_id = order_details.pizza_id
47  GROUP BY pizza_type_id
48  ORDER BY revenue DESC
49  LIMIT 3;
```

ADVANCE

PROBLEM AND QUERY

```
1      -- 1. Calculate the percentage contribution of each pizza type to total revenue.
2  ● ⊖ WITH avg_rvn AS (
3          SELECT
4              pizzas.pizza_type_id,
5              SUM(order_details.quantity * pizzas.price) AS revenue
6          FROM pizzas
7          JOIN order_details
8              ON pizzas.pizza_id = order_details.pizza_id
9          GROUP BY pizzas.pizza_type_id
10      )
11      SELECT
12          pizza_type_id,
13          ROUND(revenue * 100 / SUM(revenue) OVER (), 2)
14              AS percentage_revenue
15      FROM avg_rvn;
```

```
17      -- 2. Analyze the cumulative revenue generated over time.
18  ●   select order_date,
19      sum(revenue) over(order by order_date) as cum_revenue
20      from
21  ⊖   (select orders.order_date,
22      sum(order_details.quantity * pizzas.price) as revenue
23      from order_details join pizzas
24      on order_details.pizza_id = pizzas.pizza_id
25      join orders
26      on orders.order_id = order_details.order_id
```

PROBLEM AND QUERY

```
29  -- 3. Determine the top 3 most ordered pizza types based on revenue for each pizza category.
30  •  select name, category, revenue from
31  ⊖ (select category, name, revenue,
32     rank() over(partition by category order by revenue desc) as rn
33     from
34     ⊖ (select pizza_types.category, pizza_types.name,
35        sum(order_details.quantity * pizzas.price) as revenue
36        from pizza_types join pizzas
37        on pizza_types.pizza_type_id = pizzas.pizza_type_id
38        join order_details
39        on order_details.pizza_id = pizzas.pizza_id
40        group by pizza_types.category, pizza_types.name) as a) as b
41  where rn <= 3;
```



CONTACT INFORMATION

- khans217975@gmail.com



THANK YOU